Proyek 3 Proyek Pengembangan Perangkat Lunak Berbasis Web

Modul 3

Special Challenge

Laravel Advanced CRUD & Data Integrity



Daftar Isi

Daftar Isi 2
1. Modul Overview 3
2. Kasus dan Baseline Special Challenge 5
3. Konsep Fundamental Advanced CRUD 7
4. Eksperimen Praktikum 10
5. Special Challenge: Advanced Activity Management 14
6. Independent Challenge: Atomic Registration 16
7. Add-On : Poster Storage dan Refactoring 17
8. Quality Check dan Static Analysis 17
9. Pengumpulan dan Bukti Pengerjaan 18
10. Referensi 19

1. Modul Overview

Pada Modul 1 mahasiswa membangun struktur dan presentasi halaman menggunakan HTML dan CSS. Pada Modul 2 mahasiswa menambahkan perilaku menggunakan JavaScript, DOM, event, validasi, asynchronous JavaScript, dan debugging. Pada bagian awal Modul 3 mahasiswa telah membangun CRUD dasar Laravel dan mempelajari request lifecycle, route, controller, Form Request, service class, Eloquent, Blade, serta pemisahan tanggung jawab.

Special Challenge ini tidak bertujuan menambah banyak fitur secara acak. Fokusnya adalah meningkatkan CRUD yang sudah bekerja menjadi CRUD yang lebih dapat dipercaya: data saling berelasi dengan benar, aturan bisnis tidak mudah dilanggar, proses multi-operasi tetap konsisten, daftar data dapat dicari dan difilter secara efisien, serta siklus hidup data dapat dikelola tanpa mengorbankan keterbacaan kode.

Pertanyaan Utama Modul

Jika sebuah aplikasi sudah dapat melakukan Create, Read, Update, dan Delete, apakah aplikasi tersebut otomatis memiliki kualitas desain data dan business logic yang baik? Modul ini digunakan untuk membuktikan bahwa jawabannya bergantung pada bagaimana relasi, aturan, state, query, dan kegagalan proses ditangani.



1.1 Tujuan Pembelajaran

1. Menjelaskan perbedaan CRUD dasar dengan CRUD yang menjaga integritas dan konsistensi data.
2. Mendesain relasi one-to-many menggunakan foreign key serta menjelaskan kebijakan ketika parent masih digunakan oleh child.
3. Membedakan input validation, database constraint, dan business rule serta menempatkan masing-masing pada lapisan yang tepat.
4. Mengendalikan perubahan status entitas melalui aturan transisi yang eksplisit, bukan melalui mass assignment tanpa kontrol.
5. Menggunakan database transaction ketika satu proses bisnis terdiri atas beberapa operasi yang harus berhasil atau gagal sebagai satu kesatuan.
6. Membangun pencarian, filter, sorting, dan pagination menggunakan query database tanpa mengambil seluruh data ke memori.

7. Menggunakan query scope untuk menjaga keterbacaan dan penggunaan ulang query yang berulang.
8. Menerapkan soft delete dan restore sebagai bagian dari data lifecycle.
9. Menjelaskan masalah N+1 dan menggunakan eager loading ketika relasi memang diperlukan pada daftar data.
10. Mengelola file melalui Laravel Storage dengan validasi dan penggantian file yang terkontrol.
11. Menggunakan SonarQube untuk menemukan code smell yang relevan dan melakukan refaktorisasi berdasarkan bukti, bukan sekadar mengejar skor.
12. Menjelaskan keputusan teknis dan menunjukkan bukti pengujian setiap acceptance criterion secara mandiri.

1.2 Materi Esensial

Konsep	Makna yang Harus Dipahami
Relational data	Data aplikasi saling berhubungan. Foreign key tidak hanya menghubungkan tabel, tetapi juga membantu menjaga integritas referensial.
Data integrity	Data yang masuk dan berubah harus tetap memenuhi aturan pada level request, business process, dan database.
Business rule	Aturan domain tidak sama dengan pemeriksaan format input. Business rule menentukan tindakan yang boleh dilakukan berdasarkan keadaan data.
State transition	Status adalah bagian dari perilaku domain. Perubahan status harus mengikuti transisi yang diizinkan.
Atomic operation	Jika satu proses bisnis mempunyai beberapa operasi database, perubahan parsial harus dihindari melalui transaction.
Query responsibility	Search, filter, sort, dan pagination dibentuk menjadi query sebelum data diambil dari database.
Data lifecycle	Delete tidak selalu berarti menghilangkan data permanen. Soft delete memberikan ruang untuk restore dan audit proses.
Query efficiency	Relasi yang dipanggil berulang dapat menghasilkan N+1 query; eager loading digunakan ketika data relasi diperlukan bersama hasil utama.
Separation of concerns	Controller mengorkestrasi request; Form Request memvalidasi input; service menangani business process; model menangani data dan relasi; view menampilkan output.

1.3 Cakupan dan Batas Modul

Masuk dalam Modul	Belum Menjadi Fokus
Relationship belongsTo dan hasMany	Authentication dan session
Foreign key, unique, restrict, dan constraint dasar	Authorization, role, policy, dan permission
Advanced validation dan business rule	REST API, Sanctum, dan token
State transition dan transaction	Queue, notification, dan event-driven architecture
Search, filter, sort, pagination, query scope	Caching dan optimasi database tingkat lanjut
Soft delete, eager loading, observasi N+1	Automated testing formal; akan diperdalam pada Modul 6
File storage	Repository Pattern sebagai kewajiban arsitektur

1.4 Target Milestones

Milestone	Indikator Keberhasilan
Relational CRUD	Setiap kegiatan mempunyai kategori valid dan relasi dapat dijelaskan dari model sampai foreign key.
Integrity Guard	Kategori yang masih dipakai tidak dapat dihapus secara tidak sengaja; kode kegiatan tetap unik.
Business Rule Guard	Status kegiatan hanya berubah melalui transisi yang diizinkan dan pesan kegagalan dapat dipahami.

Milestone	Indikator Keberhasilan
Query Ready	Search, filter, sort, dan pagination bekerja bersamaan serta parameter tetap dipertahankan saat berpindah halaman.
Lifecycle Ready	Kegiatan dapat di-soft-delete dan di-restore tanpa kehilangan data relasi yang dibutuhkan.
Efficient Query	Daftar kegiatan memuat kategori menggunakan eager loading dan mahasiswa dapat menunjukkan perbedaan query dengan kasus N+1.
Quality Reviewed	Temuan SonarQube dipilah, dijelaskan, dan minimal satu refaktorisasi relevan dilakukan berdasarkan bukti.

1.5 Alur Belajar

Tahap	Aktivitas	Bukti
Predict	Tulis perilaku yang diperkirakan sebelum menjalankan perubahan.	Prediksi singkat pada worksheet.
Run	Jalankan satu perubahan atau satu skenario.	Kode, request, query, atau output.
Observe	Bandingkan hasil dengan prediksi.	Tangkapan layar, database, Console/Network, atau log query.
Explain	Jelaskan penyebab hasil dengan istilah teknis yang tepat.	Jawaban dengan kata sendiri.
Improve	Lakukan perbaikan kecil dan uji ulang skenario sebelumnya.	Commit atau versi akhir.
Checkpoint	Pastikan acceptance criteria checkpoint telah terpenuhi sebelum melanjutkan.	Checklist dan commit hash.

1.6 Ketentuan Penggunaan AI

Fase	Aktivitas Mahasiswa	Bantuan AI yang Diizinkan
Memahami brief	Membaca business rule dan menulis prediksi sendiri.	Meminta penjelasan istilah setelah mencoba merumuskan pemahaman.
Implementasi awal	Membuat migration, relationship, scope, dan method awal sendiri.	Tidak menggunakan generasi solusi lengkap untuk satu checkpoint.
Debugging	Membaca error, query, nilai aktual, dan langkah reproduksi terlebih dahulu.	Meminta satu hint diagnosis setelah bukti percobaan dicatat.
Refactoring	Menentukan masalah kode dan alternatif perbaikan.	Meminta review keterbacaan atau kemungkinan edge case.
Demonstrasi	Menjelaskan dan mengubah kode sendiri.	Tidak menggunakan bantuan AI saat demonstrasi atau code defense.

2. Kasus dan Baseline Special Challenge

Kasus yang digunakan tetap Aplikasi Manajemen Kegiatan. Mahasiswa tidak membuat proyek Laravel baru. Seluruh pekerjaan dilakukan pada repository yang telah digunakan pada CRUD dasar agar perubahan dapat dibandingkan melalui commit. Fokus perubahan berada pada kualitas model data dan perilaku aplikasi, bukan pada pergantian domain.

2.1 Baseline yang Diasumsikan Sudah Tersedia

- Project Laravel dapat dijalankan melalui local development server dan terkoneksi ke database kelas.
- Resource CRUD kegiatan telah tersedia: index, create, store, show, edit, update, dan destroy.
- Form Request dasar telah digunakan untuk input kegiatan.
- Controller tidak memuat seluruh business logic; pemisahan yang telah dibuat pada Modul 3 sebelumnya dipertahankan.
- Repository GitHub dapat diakses dan mempunyai checkpoint CRUD dasar yang bekerja sebelum special challenge dimulai.

Prinsip Baseline

Jika baseline CRUD belum bekerja, perbaiki baseline terlebih dahulu. Special Challenge tidak dinilai dari seberapa banyak kode yang ditambahkan, tetapi dari kemampuan menjaga perilaku yang sudah benar ketika model data dan aturan bisnis menjadi lebih kaya.

2.2 Model Data yang Digunakan

Entitas	Atribut Utama	Peran
Category	id, name, slug, timestamps	Mengelompokkan kegiatan. Satu kategori dapat digunakan oleh banyak kegiatan.
Activity	id, category_id, code, title, description, start_at, end_at, location, capacity, status, poster_path, timestamps, deleted_at	Entitas utama yang dikelola pada CRUD.
Registration	id, activity_id, participant_name, email, registered_at, timestamps	Digunakan pada Independent Challenge untuk menguji transaction dan konsistensi kapasitas.

2.3 Relasi dan Kebijakan Integritas

Relasi	Makna	Kebijakan
Category hasMany Activity	Satu kategori dapat memiliki banyak kegiatan.	Kategori yang masih digunakan tidak boleh dihapus.
Activity belongsTo Category	Setiap kegiatan wajib menunjuk satu kategori valid.	category_id wajib dan memiliki foreign key.
Activity hasMany Registration	Satu kegiatan dapat mempunyai banyak pendaftaran.	Pendaftaran hanya dapat dibuat untuk kegiatan yang memenuhi business rule.
Registration belongsTo Activity	Setiap pendaftaran terkait satu kegiatan.	Pendaftaran tidak boleh menunjuk activity_id yang tidak tersedia.

2.4 Status Kegiatan

Status	Makna	Transisi yang Diizinkan
draft	Data masih dapat dipersiapkan dan belum dianggap tersedia bagi peserta.	draft ke published
published	Kegiatan sudah dipublikasikan dan dapat menerima proses lanjutan.	published ke completed
completed	Kegiatan telah selesai.	Tidak ada transisi kembali pada modul ini.
deleted	Representasi lifecycle melalui soft delete, bukan nilai kolom status.	Dapat direstore sesuai kebijakan aplikasi.

2.5 Business Rules

ID	Aturan
BR-01	Setiap kegiatan wajib memiliki kategori yang tersedia pada database.
BR-02	Kode kegiatan harus unik dan tidak boleh berubah menjadi duplikat ketika data diperbarui.
BR-03	Tanggal selesai tidak boleh lebih awal daripada tanggal mulai.
BR-04	Kapasitas harus berupa bilangan positif. Batas maksimum kelas ditetapkan 500 peserta untuk latihan.
BR-05	Kegiatan hanya dapat dipublikasikan jika kategori, kode, judul, lokasi, tanggal, dan kapasitas telah lengkap dan valid.
BR-06	Transisi status yang diizinkan hanya draft ke published dan published ke completed.
BR-07	Kegiatan completed tidak dapat dikembalikan menjadi draft atau published pada modul ini.

ID	Aturan
BR-08	Kategori yang masih digunakan oleh kegiatan tidak dapat dihapus.
BR-09	Penghapusan kegiatan menggunakan soft delete sehingga data dapat dipulihkan.
BR-10	Pada Independent Challenge, pendaftaran hanya diterima jika kegiatan published, belum dimulai, kapasitas tersedia, dan email belum terdaftar pada kegiatan yang sama.

2.6 Acceptance Criteria Utama

ID	Kondisi Diterima	Bukti Minimum
AC-01	Form kegiatan hanya menerima category_id yang benar-benar tersedia.	Skenario valid dan invalid.
AC-02	Kode kegiatan unik pada create dan update.	Percobaan duplikasi.
AC-03	Kategori yang masih digunakan tidak dapat dihapus.	Error/pesan aplikasi dan data tetap ada.
AC-04	Publish gagal jika field wajib belum lengkap.	Skenario publish gagal.
AC-05	Status tidak dapat melewati urutan transisi yang ditentukan.	Percobaan transisi ilegal.
AC-06	Search berdasarkan judul/kode bekerja tanpa menghapus filter lain.	URL query dan hasil daftar.
AC-07	Filter kategori dan status dapat dipakai bersama.	Kombinasi filter.
AC-08	Sorting tanggal terbaru/terlama bekerja.	Perbandingan urutan.
AC-09	Pagination tetap membawa query parameter aktif.	Pindah halaman dengan filter tetap aktif.
AC-10	Soft delete menghilangkan item dari daftar aktif tetapi tidak menghapus record secara permanen.	Data aktif dan trashed.
AC-11	Restore mengembalikan kegiatan ke daftar aktif.	Sebelum/sesudah restore.
AC-12	Daftar kegiatan memuat kategori tanpa pola N+1 yang tidak diperlukan.	Perbandingan jumlah query.
AC-13	Controller tetap berfungsi sebagai orchestrator dan business rule tidak tersebar di Blade.	Review struktur kode.
AC-14	SonarQube dianalisis dan minimal satu temuan relevan ditindaklanjuti.	Temuan sebelum dan sesudah refactor.
AC-15	Seluruh kode dapat dijelaskan dan satu business rule dapat diubah saat demonstrasi.	Code defense.

3. Konsep Fundamental Advanced CRUD

3.1 Advance CRUD (CRUD yang dapat dipercaya)

CRUD dasar menjawab pertanyaan apakah data dapat dibuat, dibaca, diubah, dan dihapus. Advanced CRUD menambahkan pertanyaan yang lebih penting: apakah perubahan tersebut valid terhadap relasi data, aturan bisnis, keadaan entitas, dan proses lain yang terjadi bersama perubahan tersebut.

Pertanyaan CRUD Dasar	Pertanyaan Advanced CRUD
Apakah data dapat disimpan?	Apakah data boleh disimpan pada keadaan saat ini?
Apakah data dapat diubah?	Field apa yang boleh berubah dan apakah perubahan melanggar business rule?
Apakah data dapat dihapus?	Apakah data masih direferensikan? Perlu soft delete atau hard delete?
Apakah daftar dapat ditampilkan?	Bagaimana daftar dicari, difilter, diurutkan, dipaginasi, dan diambil secara efisien?
Apakah request berhasil?	Apa yang terjadi jika operasi kedua gagal setelah operasi pertama berhasil?

3.2 Relational Data dan Foreign Key

Relationship pada Eloquent memudahkan navigasi objek, sedangkan foreign key pada database menjaga agar referensi antarrecord tetap valid. Keduanya saling melengkapi dan tidak boleh dipahami sebagai hal yang sama.

Contoh relationship Eloquent

```
// app/Models/Category.php
public function activities()
{
    return $this->hasMany(Activity::class);
}

// app/Models/Activity.php
public function category()
{
    return $this->belongsTo(Category::class);
}
```

Contoh foreign key dengan kebijakan restrict

```
$table->foreignId('category_id')
    ->constrained()
    ->restrictOnDelete();
```

> **Konsep Kunci**
>
> Eloquent relationship membantu aplikasi membaca hubungan data. Foreign key membantu database menolak referensi yang tidak valid. Kebijakan restrictOnDelete dipilih karena kategori yang masih digunakan tidak boleh hilang dan meninggalkan kegiatan tanpa konteks.

3.3 Validation Rule, Database Constraint, dan Business Rule

Lapisan	Contoh	Tujuan
Form Request	required, date, integer, exists, unique	Menyaring input request sebelum memasuki proses aplikasi.
Database constraint	foreign key, unique index, NOT NULL	Menjadi penjaga terakhir agar data yang disimpan tetap konsisten.
Business rule	kegiatan completed tidak boleh kembali ke draft	Menentukan tindakan yang diperbolehkan berdasarkan makna dan state domain.

Aturan yang sama tidak selalu perlu ditulis di semua lapisan. Pilih penjagaan yang sesuai. Sebagai contoh, keunikan kode layak diperiksa pada Form Request untuk umpan balik pengguna dan tetap dijaga oleh unique index agar database tidak menerima duplikasi dari jalur lain.

3.4 State dan Transition

Kolom status bukan sekadar teks. Status menentukan tindakan yang diperbolehkan. Karena itu, update status sebaiknya tidak bergantung pada input bebas dari form umum. Gunakan method atau service yang secara eksplisit memeriksa state awal dan target state.

Contoh guard clause pada service

```
public function publish(Activity $activity): Activity
{
    if ($activity->status !== 'draft') {
        throw ValidationException::withMessages([
            'status' => 'Hanya kegiatan draft yang dapat dipublikasikan.',
        ]);
    }

    // Pemeriksaan kelengkapan business rule dilakukan di sini.

    $activity->update(['status' => 'published']);
    return $activity;
```

```
}
```

Yang Harus Dapat Dijelaskan

Mengapa perubahan status tidak cukup dilakukan dengan $activity->update($request->all())? Karena mass assignment tidak menjelaskan transisi yang diizinkan, tidak melindungi invariants domain, dan membuat business rule mudah tersebar di banyak tempat.

3.5 Atomic Operation dan Database Transaction

Transaction digunakan ketika satu proses bisnis memiliki lebih dari satu perubahan database yang harus dianggap satu kesatuan. Jika salah satu operasi gagal, operasi sebelumnya harus dibatalkan sehingga aplikasi tidak meninggalkan data parsial.

Pola transaction pada proses pendaftaran

```
DB::transaction(function () use ($activity, $data) {
    $registration = $activity->registrations()->create($data);
    $activity->increment('registered_count');

    return $registration;
});
```

Contoh di atas digunakan pada Independent Challenge. Penyimpanan registered_count dipakai secara sengaja untuk mendemonstrasikan konsistensi multi-operasi. Persoalan race condition dan locking tingkat lanjut berada di luar cakupan modul ini.

3.6 Search, Filter, Sort, dan Pagination

Fitur daftar sebaiknya membentuk query sesuai kebutuhan sebelum data diambil. Mengambil semua record lalu memfilter dengan collection di PHP tidak sesuai untuk data yang bertambah karena memindahkan pekerjaan dan penggunaan memori ke aplikasi.

Pola query bertahap

```
$activities = Activity::query()
    ->with('category')
    ->when($request->search, fn ($q, $search) =>
        $q->where(function ($q) use ($search) {
            $q->where('title', 'like', "%{$search}%")
              ->orWhere('code', 'like', "%{$search}%");
        })
    )
    ->when($request->category_id, fn ($q, $id) =>
        $q->where('category_id', $id)
    )
    ->paginate(10)
    ->withQueryString();
```

3.7 Query Scope sebagai Abstraksi Query

Jika aturan query yang sama mulai digunakan di lebih dari satu tempat, local scope membantu memberi nama pada maksud query. Scope bukan tempat seluruh business process, tetapi cocok untuk ekspresi query yang berulang dan spesifik terhadap model.

Contoh local scope

```
public function scopeSearch($query, ?string $keyword)
{
    return $query->when($keyword, function ($query, $keyword) {
        $query->where(function ($query) use ($keyword) {
            $query->where('title', 'like', "%{$keyword}%")
                  ->orWhere('code', 'like', "%{$keyword}%");
        });
    });
}
```

3.8 Soft Delete dan Data Lifecycle

Soft delete menyimpan deleted_at daripada menghapus record permanen. Tujuannya bukan agar semua tabel wajib memakai soft delete, tetapi agar mahasiswa mempertimbangkan lifecycle data. Pada kasus kegiatan, restore dibutuhkan karena penghapusan dapat terjadi akibat kesalahan operasional.

Contoh SoftDeletes

```
use Illuminate\Database\Eloquent\SoftDeletes;

class Activity extends Model
{
    use SoftDeletes;
}

// Query khusus data terhapus
Activity::onlyTrashed()->latest()->paginate(10);
```

3.9 N+1 dan Eager Loading

N+1 terjadi ketika daftar utama menghasilkan satu query, kemudian setiap item memicu query tambahan untuk relasi. Gejalanya mudah terlewat karena tampilan tetap benar. Masalah baru terlihat pada jumlah query dan waktu akses ketika data bertambah.

Pendekatan	Karakteristik
Activity::all() lalu membaca $activity->category pada loop	Berpotensi menghasilkan satu query daftar dan query kategori untuk setiap item.
Activity::with('category')->get()	Relasi kategori dimuat bersama kebutuhan daftar sehingga jumlah query tidak bertambah per item.

3.10 File Storage

Laravel Storage memisahkan lokasi penyimpanan file dari path yang disimpan pada database. Mahasiswa harus memvalidasi file, menyimpan path, dan menangani penggantian file lama secara sadar.

Contoh penyimpanan file

```
$path = $request->file('poster')->store('posters', 'public');
$activity->update(['poster_path' => $path]);
```

Batas Keamanan

Database menyimpan path atau identifier file, bukan isi file biner. Jangan menyimpan file sensitif pada public disk. Modul ini hanya menggunakan poster kegiatan sebagai contoh file publik.

4. Eksperimen Praktikum

Eksperimen dilakukan sebelum Special Challenge agar mahasiswa memiliki bukti kecil untuk setiap konsep. Kerjakan pada branch khusus, misalnya modul3-special-challenge. Setelah eksperimen selesai, pulihkan data uji yang sengaja dibuat rusak.

4.1 Eksperimen 1 - Relationship dan Referential Integrity

TUJUAN

Membuktikan hubungan antara Eloquent relationship, foreign key, dan kebijakan penghapusan kategori yang masih digunakan.

BAHAN AWAL

```
php artisan make:model Category -m
php artisan make:migration add_category_id_to_activities_table --table=activities
```

```
Schema::create('categories', function (Blueprint $table) {
    $table->id();
    $table->string('name');
```

```
    $table->string('slug')->unique();
    $table->timestamps();
});

Schema::table('activities', function (Blueprint $table) {
    $table->foreignId('category_id')
        ->after('id')
        ->constrained()
        ->restrictOnDelete();
});
```

LANGKAH KERJA

1. Sebelum menjalankan migration, prediksi apa yang terjadi jika tabel activities telah memiliki data tetapi category_id langsung dibuat wajib. Jelaskan strategi migrasi data yang aman untuk kondisi tersebut.
2. Untuk database latihan yang dapat di-reset, lakukan migrate:fresh dan siapkan minimal dua kategori serta tiga kegiatan.
3. Tambahkan relationship category pada Activity dan activities pada Category.
4. Dari Tinker, ambil satu Activity lalu akses nama kategori melalui relationship.
5. Coba membuat Activity menggunakan category_id yang tidak tersedia. Catat apakah kegagalan terjadi pada aplikasi atau database.
6. Coba menghapus kategori yang masih digunakan. Catat pesan error atau exception yang dihasilkan, lalu buat penanganan aplikasi yang memberikan pesan lebih jelas.
7. Ulangi penghapusan pada kategori yang tidak digunakan dan bandingkan hasilnya.

Skenario	Prediksi	Hasil Aktual	Bukti/Penjelasan
Activity dengan kategori valid			
Activity dengan category_id tidak tersedia			
Hapus kategori yang masih digunakan			
Hapus kategori tanpa kegiatan			

PERTANYAAN PEMAHAMAN

1. Apa perbedaan fungsi relationship Eloquent dan foreign key database?
2. Mengapa restrictOnDelete lebih sesuai daripada cascadeOnDelete untuk aturan kategori pada kasus ini?
3. Jika foreign key sudah tersedia, mengapa Form Request tetap perlu memeriksa exists:categories,id?

4.2 Eksperimen 2 - Advanced Validation dan Business Rule

TUJUAN

Membedakan aturan yang dapat divalidasi langsung dari request dengan aturan yang bergantung pada state dan proses domain.

Contoh rule input

```
public function rules(): array
{
    return [
        'category_id' => ['required', 'exists:categories,id'],
        'code' => ['required', 'string', 'max:30'],
        'title' => ['required', 'string', 'max:150'],
        'start_at' => ['required', 'date'],
        'end_at' => ['required', 'date', 'after_or_equal:start_at'],
        'capacity' => ['required', 'integer', 'min:1', 'max:500'],
    ];
}
```

LANGKAH KERJA

1. Identifikasi BR-01 sampai BR-07. Tandai rule yang cukup ditangani pada Form Request dan rule yang memerlukan state Activity.
2. Uji create dengan end_at lebih awal daripada start_at dan buktikan request ditolak.

3. Uji kode duplikat. Pada update, pastikan record yang sedang diedit tidak dianggap duplikat terhadap dirinya sendiri.
4. Buat method publish pada service. Method harus menolak Activity yang bukan draft.
5. Tambahkan pemeriksaan kelengkapan field yang diwajibkan sebelum publish.
6. Coba mengubah completed langsung menjadi draft melalui request umum. Pastikan aplikasi tidak mengizinkan perubahan tersebut.
7. Catat alasan mengapa transisi status ditempatkan pada method khusus.

Rule	Form Request	DB Constraint	Business Service	Alasan
category_id harus tersedia				
code unik				
end_at >= start_at				
draft dapat publish				
completed tidak dapat kembali ke draft				

4.3 Eksperimen 3 - Search, Filter, Sort, dan Pagination

TUJUAN

Menyusun query berdasarkan parameter request serta menjaga parameter aktif ketika pengguna berpindah halaman.

DATASET

Siapkan minimal 15 kegiatan dari dua kategori dan minimal dua status. Data dapat dibuat menggunakan seeder agar pengujian dapat diulang.

1. Tambahkan input search untuk code atau title.
2. Tambahkan filter category_id dan status.
3. Tambahkan pilihan sort: terbaru dan terlama berdasarkan start_at.
4. Gunakan paginate(10), bukan get(), untuk daftar utama.
5. Gunakan withQueryString() agar search/filter/sort tidak hilang pada link pagination.
6. Uji kombinasi search + category + status + sort, lalu pindah ke halaman berikutnya.
7. Refaktor kondisi query yang berulang menjadi local scope jika mulai mengganggu keterbacaan controller.

Skenario	Query Parameter	Ekspektasi	Hasil
Search judul			
Filter kategori + status			
Sort terbaru			
Pagination dengan filter aktif			
Kombinasi seluruh parameter			

4.4 Eksperimen 4 - Soft Delete dan Restore

TUJUAN

Membedakan penghapusan logis dari penghapusan permanen dan menguji lifecycle data kegiatan.

1. Tambahkan softDeletes() pada migration activities dan gunakan trait SoftDeletes pada model.
2. Jalankan destroy pada satu kegiatan draft lalu buktikan record tidak tampil pada query normal.
3. Dari Tinker, bandingkan Activity::count(), Activity::withTrashed()->count(), dan Activity::onlyTrashed()->count().
4. Buat halaman atau aksi restore sederhana untuk data yang terhapus.
5. Restore record dan pastikan ia kembali ke daftar aktif.
6. Jelaskan kondisi apa yang seharusnya menggunakan forceDelete dan mengapa fungsi tersebut tidak diberikan sebagai aksi utama pada UI modul ini.

Pemeriksaan	Sebelum Delete	Sesudah Soft Delete	Sesudah Restore
Tampil pada index			

Pemeriksaan	Sebelum Delete	Sesudah Soft Delete	Sesudah Restore
Tersedia di onlyTrashed			
Record tetap ada di DB			

4.5 Eksperimen 5 - Membuktikan N+1

TUJUAN

Menggunakan bukti jumlah query untuk membedakan lazy loading berulang dan eager loading.

Percobaan A - tanpa eager loading

```
DB::enableQueryLog();

$activities = Activity::take(10)->get();
foreach ($activities as $activity) {
    $activity->category?->name;
}

count(DB::getQueryLog());
```

Percobaan B - dengan eager loading

```
DB::flushQueryLog();
DB::enableQueryLog();

$activities = Activity::with('category')->take(10)->get();
foreach ($activities as $activity) {
    $activity->category?->name;
}

count(DB::getQueryLog());
```

1. Jalankan kedua percobaan menggunakan dataset yang sama.
2. Catat jumlah query dan jelaskan mengapa tampilan dapat sama tetapi biaya query berbeda.
3. Tambahkan with(category) pada query index hanya jika kategori memang digunakan oleh view daftar.
4. Jelaskan mengapa eager loading semua relationship tanpa kebutuhan juga bukan kebiasaan yang baik.

Pendekatan	Jumlah Query	Kesimpulan
Lazy loading pada loop		
Eager loading with(category)		

4.6 Eksperimen 6 - Transaction dan Rollback Terkontrol

TUJUAN

Membuktikan bahwa transaction membatalkan seluruh perubahan ketika terjadi exception sebelum proses selesai.

Percobaan rollback

```
DB::transaction(function () {
    $category = Category::create([
        'name' => 'Eksperimen Transaction',
        'slug' => 'eksperimen-transaction',
    ]);

    Activity::create([
        // Isi field valid menggunakan $category->id
    ]);

    throw new RuntimeException('Simulasi kegagalan.');
});
```

1. Sebelum menjalankan kode, prediksi apakah Category dan Activity tetap tersimpan setelah exception.
2. Jalankan percobaan pada Tinker atau route khusus sementara.
3. Periksa database setelah exception.

4. Ulangi percobaan tanpa exception dan bandingkan hasil.
5. Hapus route atau kode eksperimen setelah bukti dikumpulkan.

Inti Eksperimen

Transaction bukan mekanisme untuk menyembunyikan error. Error tetap harus ditangani. Fungsi transaction adalah menjaga agar perubahan database tidak berhenti pada keadaan setengah selesai.

5. Special Challenge: Advanced Activity Management

Special Challenge terdiri atas tiga task yang menggunakan repository, domain, dan dataset yang sama. Mahasiswa tidak membuat aplikasi terpisah untuk setiap task. Commit setiap checkpoint agar proses perubahan dapat ditelusuri.

Komponen	Bobot	Waktu Disarankan	Hasil Utama
Task 1 - Relational CRUD dan Integrity	25%	45 menit	Kategori terhubung ke kegiatan; foreign key, unique, dan delete policy teruji.
Task 2 - Business Rule dan Query Experience	35%	70 menit	State transition, search, filter, sort, dan pagination bekerja sesuai acceptance criteria.
Task 3 - Data Lifecycle dan Query Quality	25%	45 menit	Soft delete, restore, eager loading, dan quality review diterapkan.
Add On - File Storage dan Refactoring	15%	Di luar sesi	Poster dikelola melalui Storage dan refaktorisasi terdokumentasi.

5.1 Task 1 - Relational CRUD dan Integrity

TARGET

Mengubah CRUD kegiatan dari entitas tunggal menjadi CRUD yang terhubung dengan kategori dan dilindungi constraint database.

PEKERJAAN

1. Buat migration, model, dan seeder Category.
2. Tambahkan category_id pada Activity dan definisikan relationship dua arah.
3. Pastikan code pada Activity mempunyai unique index.
4. Perbarui create dan edit form agar kategori dipilih dari data Category.
5. Gunakan validation rule exists untuk category_id dan unique untuk code pada create/update.
6. Terapkan kebijakan bahwa kategori yang masih digunakan tidak dapat dihapus. Berikan pesan aplikasi yang dapat dipahami pengguna.
7. Tampilkan nama kategori pada index dan detail Activity menggunakan relationship.
8. Uji minimal enam skenario pada tabel berikut.

Skenario	Hasil yang Diharapkan	Status	Bukti
Create dengan kategori valid	Data tersimpan dan relasi benar		
Create dengan category_id tidak valid	Request ditolak		
Create kode duplikat	Request ditolak		
Update tanpa mengganti kode	Tidak dianggap duplikat terhadap diri sendiri		
Hapus kategori yang masih dipakai	Ditolak; data kegiatan tetap utuh		
Hapus kategori kosong	Berhasil		

KRITERIA SELESAI

[ ] Relationship model dan foreign key dapat dijelaskan.
[ ] Tidak ada Activity dengan category_id yatim.
[ ] Kode kegiatan unik dijaga di request dan database.
[ ] Delete policy sesuai BR-08.

[ ] Controller tidak berisi query kategori atau business rule yang berulang tanpa alasan.

5.2 Task 2 - Business Rule dan Query Experience

TARGET

Membangun daftar kegiatan yang dapat digunakan secara realistis sekaligus mengendalikan status melalui business rule yang eksplisit.

PEKERJAAN

1. Pastikan status awal Activity adalah draft.
2. Buat aksi publish yang hanya menerima Activity draft dan memenuhi BR-05.
3. Buat aksi complete yang hanya menerima Activity published.
4. Jangan menyediakan update status bebas pada form edit umum.
5. Tambahkan search berdasarkan code atau title.
6. Tambahkan filter category_id dan status.
7. Tambahkan sort berdasarkan start_at terbaru atau terlama.
8. Gunakan pagination 10 item dan pertahankan query string aktif.
9. Gunakan local scope apabila kondisi search/filter sudah mulai berulang atau mengganggu keterbacaan controller.
10. Uji kombinasi filter, bukan hanya satu parameter secara terpisah.

Skenario	Hasil yang Diharapkan	Bukti
Draft lengkap -> Published	Berhasil dan status berubah	
Draft tidak lengkap -> Published	Ditolak dengan pesan yang jelas	
Published -> Completed	Berhasil	
Completed -> Draft	Ditolak	
Search + Category + Status	Hasil memenuhi seluruh parameter	
Pagination halaman 2	Parameter pencarian/filter tetap ada	

PERTANYAAN

1. Mengapa status tidak diubah melalui form edit umum?
2. Bagian mana yang merupakan validation rule dan bagian mana yang merupakan business rule?
3. Mengapa filter dilakukan di query database dan bukan setelah Activity::all()?
4. Kapan local scope membantu dan kapan justru membuat model terlalu penuh?

5.3 Task 3 - Data Lifecycle dan Query Quality

TARGET

Menambahkan soft delete dan restore, kemudian membuktikan bahwa daftar kegiatan tidak menghasilkan N+1 query untuk kategori.

1. Tambahkan deleted_at dan trait SoftDeletes pada Activity.
2. Ubah destroy agar melakukan soft delete.
3. Buat daftar sederhana Trash atau halaman data terhapus.
4. Sediakan aksi restore.
5. Pastikan index normal tidak menampilkan data yang terhapus.
6. Gunakan eager loading category pada query index.
7. Gunakan eksperimen query log untuk membuktikan perubahan jumlah query sebelum dan sesudah eager loading.
8. Jalankan SonarQube setelah Task 1-3 dan catat temuan yang relevan dengan kode yang baru ditambahkan.

Acceptance Criterion	Status	Bukti
AC-10 Soft delete		
AC-11 Restore		
AC-12 Tidak ada pola N+1 pada index		

Acceptance Criterion	Status	Bukti
AC-13 Struktur tanggung jawab tetap jelas		
AC-14 Satu refaktorisasi berbasis static analysis		

6. Independent Challenge: Atomic Registration

Independent Challenge digunakan untuk memastikan mahasiswa memahami transaction pada proses bisnis nyata. Challenge ini tidak boleh dikerjakan dengan menyalin solusi lengkap. Gunakan hint bertahap setelah percobaan sendiri dicatat pada worksheet.

6.1 Skenario

Tambahkan fitur pendaftaran peserta sederhana untuk Activity. Fitur tidak menggunakan authentication karena authentication berada pada modul berikutnya. Pengguna mengisi nama dan email pada Activity yang published.

Aturan	Keterangan
IC-01	Pendaftaran hanya untuk Activity berstatus published.
IC-02	Pendaftaran ditolak jika start_at sudah lewat.
IC-03	Email yang sama tidak boleh mendaftar dua kali pada Activity yang sama.
IC-04	Jumlah pendaftar tidak boleh melebihi capacity.
IC-05	Satu proses pendaftaran membuat Registration dan memperbarui registered_count sebagai satu transaction.
IC-06	Jika update registered_count gagal, Registration tidak boleh tertinggal sebagai data parsial.

6.2 Struktur Data Tambahan

Migration untuk latihan atomic registration

```
Schema::create('registrations', function (Blueprint $table) {
    $table->id();
    $table->foreignId('activity_id')->constrained();
    $table->string('participant_name');
    $table->string('email');
    $table->timestamp('registered_at');
    $table->timestamps();

    $table->unique(['activity_id', 'email']);
});

Schema::table('activities', function (Blueprint $table) {
    $table->unsignedInteger('registered_count')->default(0);
});
```

Catatan Desain

Kolom registered_count sengaja digunakan agar mahasiswa mempunyai dua operasi database yang harus konsisten. Pada sistem lain, jumlah peserta dapat dihitung langsung dari relation. Modul ini menggunakan denormalisasi tersebut sebagai sarana memahami transaction; konsekuensi desain harus dapat dijelaskan.

6.3 Hint Bertahap

Level	Hint
Hint 1	Pisahkan pemeriksaan rule dari penyimpanan. Pastikan Activity memenuhi status, waktu, dan kapasitas sebelum memulai perubahan.
Hint 2	Gunakan unique index gabungan activity_id dan email sebagai penjaga database selain validasi aplikasi.
Hint 3	Tempatkan create Registration dan increment registered_count di dalam DB::transaction.
Hint 4	Untuk membuktikan rollback, buat kegagalan terkontrol di antara create dan increment, lalu periksa apakah Registration tertinggal.

6.4 Bukti Wajib

[ ] Satu pendaftaran valid berhasil.
[ ] Email duplikat pada kegiatan yang sama ditolak.
[ ] Kegiatan draft menolak pendaftaran.
[ ] Kapasitas penuh menolak pendaftaran.
[ ] Eksperimen kegagalan terkontrol membuktikan rollback.
[ ] Mahasiswa dapat menjelaskan mengapa validation saja tidak menggantikan transaction.

7. Add-On : Poster Storage dan Refactoring

7.1 Fitur Poster Kegiatan

1. Tambahkan input poster opsional pada create dan edit.
2. Validasi file sebagai image dengan batas ukuran yang wajar, misalnya maksimum 2 MB.
3. Simpan file pada public disk menggunakan Laravel Storage.
4. Simpan path file pada poster_path, bukan isi file.
5. Ketika poster diganti, hapus file lama setelah file baru berhasil tersimpan.
6. Ketika Activity di-soft-delete, file poster tidak langsung dihapus agar restore tetap dapat menampilkan poster.
7. Definisikan kebijakan ketika force delete dilakukan secara manual oleh administrator di masa mendatang; implementasi force delete tidak wajib pada modul ini.

7.2 Refactoring Review

Pemeriksaan	Pertanyaan Review
Controller	Apakah controller masih berfungsi sebagai orchestrator atau mulai memuat business rule/query detail berlebihan?
Form Request	Apakah input validation terpusat dan pesan kesalahan dapat dipahami?
Service	Apakah state transition dan proses domain memiliki method yang jelas?
Model	Apakah relationship dan query scope relevan terhadap model, bukan sekadar memindahkan semua kode ke model?
Blade	Apakah Blade menampilkan data tanpa menjalankan business rule atau query database?
Penamaan	Apakah nama method menggambarkan tindakan dan menghindari istilah generik seperti processData?
Duplikasi	Apakah kondisi search/filter atau rule yang sama ditulis berulang?

7.3 Definition of Done Add-On

[ ] Poster valid dapat diunggah dan ditampilkan.
[ ] File yang tidak sesuai ditolak dengan pesan yang jelas.
[ ] Penggantian poster tidak meninggalkan file lama tanpa alasan.
[ ] Soft delete tidak merusak kemungkinan restore poster.
[ ] Minimal satu refaktorisasi keterbacaan dilakukan dan dijelaskan pada README/worksheet.
[ ] Commit Add-On terpisah dari checkpoint Task 1-3.

8. Quality Check dan Static Analysis

Static analysis digunakan sebagai alat observasi. Mahasiswa tidak dinilai dari angka SonarQube yang paling tinggi, tetapi dari kemampuan membaca temuan, menentukan relevansi, memperbaiki masalah yang memang penting, dan membuktikan bahwa fitur tetap bekerja setelah refaktorisasi.

8.1 Prosedur Quality Check

1. Pastikan aplikasi bekerja pada skenario normal sebelum static analysis.
2. Jalankan analisis SonarQube sesuai konfigurasi kelas.
3. Pilih minimal tiga temuan yang berada pada kode mahasiswa, bukan vendor.
4. Kelompokkan temuan menjadi relevan, perlu konteks, atau tidak relevan terhadap latihan.

5. Perbaiki minimal satu temuan yang jelas meningkatkan maintainability atau mencegah kesalahan.
6. Ulangi skenario acceptance criteria yang berkaitan dengan kode yang diubah.
7. Catat before/after pada worksheet.

Temuan	Mengapa Terjadi	Keputusan	Perubahan	Bukti Uji Ulang

8.2 Checklist Clean Code

[ ] Controller tidak berisi business rule status yang tersebar.
[ ] Form Request menangani input validation, bukan proses bisnis.
[ ] Service method memiliki tujuan yang spesifik.
[ ] Query list tidak menggunakan Activity::all() lalu memfilter collection untuk fitur search/filter.
[ ] Relationship diberi nama berdasarkan domain.
[ ] Query scope digunakan jika benar-benar memperbaiki keterbacaan atau reuse.
[ ] Tidak ada inner query/akses relationship yang menyebabkan N+1 pada index.
[ ] Error tidak disembunyikan; pesan pengguna dan bukti teknis dapat dibedakan.
[ ] Tidak ada kode eksperimen sementara yang tertinggal pada branch final.

9. Pengumpulan dan Bukti Pengerjaan

9.1 Struktur Repository yang Disarankan

```
app/
|-- Http/
|   |-- Controllers/
|   `-- Requests/
|-- Models/
`-- Services/

database/
|-- migrations/
|-- seeders/
`-- factories/          # bila digunakan

resources/views/
routes/web.php
README.md
```

9.2 Checkpoint Git

Checkpoint	Isi Minimum	Contoh Tag/Commit
Baseline	CRUD dasar sebelum challenge	modul3-crud-baseline
Task 1	Relationship dan integrity	modul3-special-task1
Task 2	Business rule dan query experience	modul3-special-task2
Task 3	Soft delete dan query quality	modul3-special-task3
Add-on	Poster storage dan refactoring	modul3-special-final

9.3 Bukti Minimum pada Worksheet

- Diagram atau tabel relationship Category-Activity-Registration.
- Hasil migration/struktur database yang menunjukkan foreign key dan unique constraint.
- Bukti penghapusan kategori yang masih digunakan ditolak.
- Bukti transisi status valid dan tidak valid.
- Bukti kombinasi search/filter/sort/pagination.
- Bukti soft delete dan restore.

- Perbandingan query lazy loading dan eager loading.
- Bukti rollback transaction pada Independent Challenge.
- Hasil SonarQube dan catatan keputusan refaktorisasi.
- URL repository dan commit/tag setiap checkpoint.

9.4. Lesson Learnt

1. Konsep apa yang paling mengubah cara Anda melihat CRUD setelah challenge ini?
2. Business rule mana yang paling sulit ditempatkan dan mengapa?
3. Apa perbedaan kegagalan validation dengan kegagalan database constraint yang Anda amati?
4. Apa bukti yang membuat Anda memahami transaction atau rollback?
5. Apa yang Anda pelajari dari perbandingan query sebelum dan sesudah eager loading?
6. Temuan SonarQube apa yang benar-benar membantu dan temuan apa yang memerlukan konteks?
7. Jika aplikasi ini dilanjutkan ke authentication dan authorization, bagian mana yang menurut Anda akan berubah pada modul berikutnya?

10. Referensi

1. Laravel Documentation. Eloquent: Relationships. https://laravel.com/docs/eloquent-relationships
2. Laravel Documentation. Validation. https://laravel.com/docs/validation
3. Laravel Documentation. Database: Transactions. https://laravel.com/docs/database#database-transactions
4. Laravel Documentation. Eloquent: Query Scopes. https://laravel.com/docs/eloquent#query-scopes
5. Laravel Documentation. Pagination. https://laravel.com/docs/pagination
6. Laravel Documentation. Eloquent: Soft Deleting. https://laravel.com/docs/eloquent#soft-deleting
7. Laravel Documentation. File Storage. https://laravel.com/docs/filesystem
8. Laravel Documentation. Database: Running Database Queries. https://laravel.com/docs/database
9. SonarSource Documentation. SonarQube Server / Community Build Documentation. https://docs.sonarsource.com/sonarqube-server/
10. Martin Fowler. Patterns of Enterprise Application Architecture. Addison-Wesley.
11. Robert C. Martin. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.